ID: 314561005
Name: 龍偉亮

Lab 1 Report

Machine: Ubuntu 26.04, driver 595.91.07, CUDA 13.1 (nvcc 13.1.115), built with `-arch=native` (sm_89).

Part 0: Know your GPU

Q1. GPU details and peaks

Item	Value	Source
GPU	NVIDIA GeForce RTX 4060 Ti (8 GB), AD106	`nvidia-smi`
Compute capability	8.9	`nvidia-smi --query-gpu=compute_cap` , harness reports `sm_89`
SM count	34	harness header (`cudaGetDeviceProperties`)
CUDA cores	4352 (34 SMs x 128)	TechPowerUp GPU Database
Boost clock	2.535 GHz	TechPowerUp GPU Database
Memory	18 Gbps GDDR6, 128-bit bus	TechPowerUp GPU Database

> The spec-sheet values (4352 shading units, 2535 MHz boost, 2250 MHz memory clock = 18 Gbps GDDR6, 128-bit bus, AD106) are from the TechPowerUp entry for the GeForce RTX 4060 Ti 8 GB. The 34 SM count and CC 8.9 were read from the device.

Peak FP32 = 2 x CUDA cores x boost clock = 2 x 4352 x 2.535 GHz = **22,065 GFLOP/s** (about 22.06 TFLOP/s).

Peak DRAM bandwidth = data rate x bus width / 8 = 18 Gbps x 128 / 8 = **288 GB/s**.

The harness prints its own estimate of 22,326 GFLOP/s. It is derived from the 2.565 GHz clock the driver reports, not the spec-sheet boost clock, so the spec-sheet 22,065 GFLOP/s is used for all percentages below.

Q2. Ridge point and arithmetic intensity

- Ridge point = peak FLOP/s / peak bytes/s = 22,065 / 288 = **76.6 FLOP/byte**.
- SGEMM at M = N = K = 4096: FLOPs = 2N^3. Minimum DRAM traffic = read A + read B + write C = 3 x N^2 x 4 bytes = 12N^2 bytes.
  Arithmetic intensity = 2N^3 / 12N^2 = N/6 = 4096/6 = **about 683 FLOP/byte**.
- 683 is about 9x the ridge point (76.6), so SGEMM at this size should be **compute-bound** on this GPU. A good kernel is limited by FP32 throughput, not by DRAM bandwidth.
  The naive kernel below is far below this limit, because it is limited by something else (L1 access), not by DRAM.

Part 1: The naive kernel and the profiler

Q1. Benchmark results (naive vs cuBLAS)

Commands: `./sgemm naive S S S` and `./sgemm cublas S S S` for S = 1024, 2048, 4096. All runs PASS.

Size	naive ms	naive GFLOP/s	cuBLAS ms	cuBLAS GFLOP/s	naive / cuBLAS
1024^3	13.24	162.2	0.183	11,716	1.4%
2048^3	93.94	182.9	1.317	13,042	1.4%

Size	naive ms	naive GFLOP/s	cuBLAS ms	cuBLAS GFLOP/s	naive / cuBLAS
4096^3	748.82	183.5	10.923	12,582	1.5%

```
GPU: NVIDIA GeForce RTX 4060 Ti  (sm_89, 34 SMs, L2 32.0 MB)
Approx. peaks: FP32 22326 GFLOP/s, DRAM 288 GB/s  (verify against your spec sheet!)
Problem: M=1024 N=1024 K=1024 alpha=1 beta=0

id  kernel                          ms    GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
1   naive                      13.2359      162.2       1.0     1.4%     0.7%     0.3%  PASS (err 2.3e-05)
GPU: NVIDIA GeForce RTX 4060 Ti  (sm_89, 34 SMs, L2 32.0 MB)
Approx. peaks: FP32 22326 GFLOP/s, DRAM 288 GB/s  (verify against your spec sheet!)
Problem: M=2048 N=2048 K=2048 alpha=1 beta=0

id  kernel                          ms    GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
1   naive                      93.9377      182.9       0.5     1.4%     0.8%     0.2%  PASS (err 5.3e-05)
GPU: NVIDIA GeForce RTX 4060 Ti  (sm_89, 34 SMs, L2 32.0 MB)
Approx. peaks: FP32 22326 GFLOP/s, DRAM 288 GB/s  (verify against your spec sheet!)
Problem: M=4096 N=4096 K=4096 alpha=1 beta=0

id  kernel                          ms    GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
1   naive                     748.8174      183.5       0.3     1.5%     0.8%     0.1%  PASS (err 0.0e+00)
GPU: NVIDIA GeForce RTX 4060 Ti  (sm_89, 34 SMs, L2 32.0 MB)
Approx. peaks: FP32 22326 GFLOP/s, DRAM 288 GB/s  (verify against your spec sheet!)
Problem: M=1024 N=1024 K=1024 alpha=1 beta=0

id  kernel                          ms    GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
0   cublas                      0.1833    11715.9      68.6   100.0%    52.5%    23.8%  PASS (err 0.0e+00)
GPU: NVIDIA GeForce RTX 4060 Ti  (sm_89, 34 SMs, L2 32.0 MB)
Approx. peaks: FP32 22326 GFLOP/s, DRAM 288 GB/s  (verify against your spec sheet!)
Problem: M=2048 N=2048 K=2048 alpha=1 beta=0

id  kernel                          ms    GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
0   cublas                      1.3173    13041.9      38.2   100.0%    58.4%    13.3%  PASS (err 0.0e+00)
GPU: NVIDIA GeForce RTX 4060 Ti  (sm_89, 34 SMs, L2 32.0 MB)
Approx. peaks: FP32 22326 GFLOP/s, DRAM 288 GB/s  (verify against your spec sheet!)
Problem: M=4096 N=4096 K=4096 alpha=1 beta=0

id  kernel                          ms    GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
0   cublas                     10.9232    12582.3      18.4   100.0%    56.4%     6.4%  PASS (err 0.0e+00)
```

Q2. Is the naive kernel limited by memory or by FP32 math?

Profile: `scripts/profile.sh naive 2048 2048 2048` (ncu `--set full`, `--clock-control none`).
The kernel runs in 93.96 ms.

GPU Speed Of Light Throughput:

Metric	Value
Memory Throughput	**98.55%**
L1/TEX Cache Throughput	99.02%
L2 Cache Throughput	2.32%
DRAM Throughput	0.66%
Compute (SM) Throughput	**11.95%**

Memory Workload Analysis:

- L1/TEX hit rate 99.2%, L2 hit rate 94.2%.
- Global loads: 8,862,564,352 sectors / 537,001,984 requests = **16.5 sectors per request**.

Warp State Statistics:

- 192.1 cycles between issued instructions per warp.
- The dominant stall is **LG throttle** (waiting for the L1 queue for global memory operations to have room): 180.4 of 192.1 cycles (93.9%).

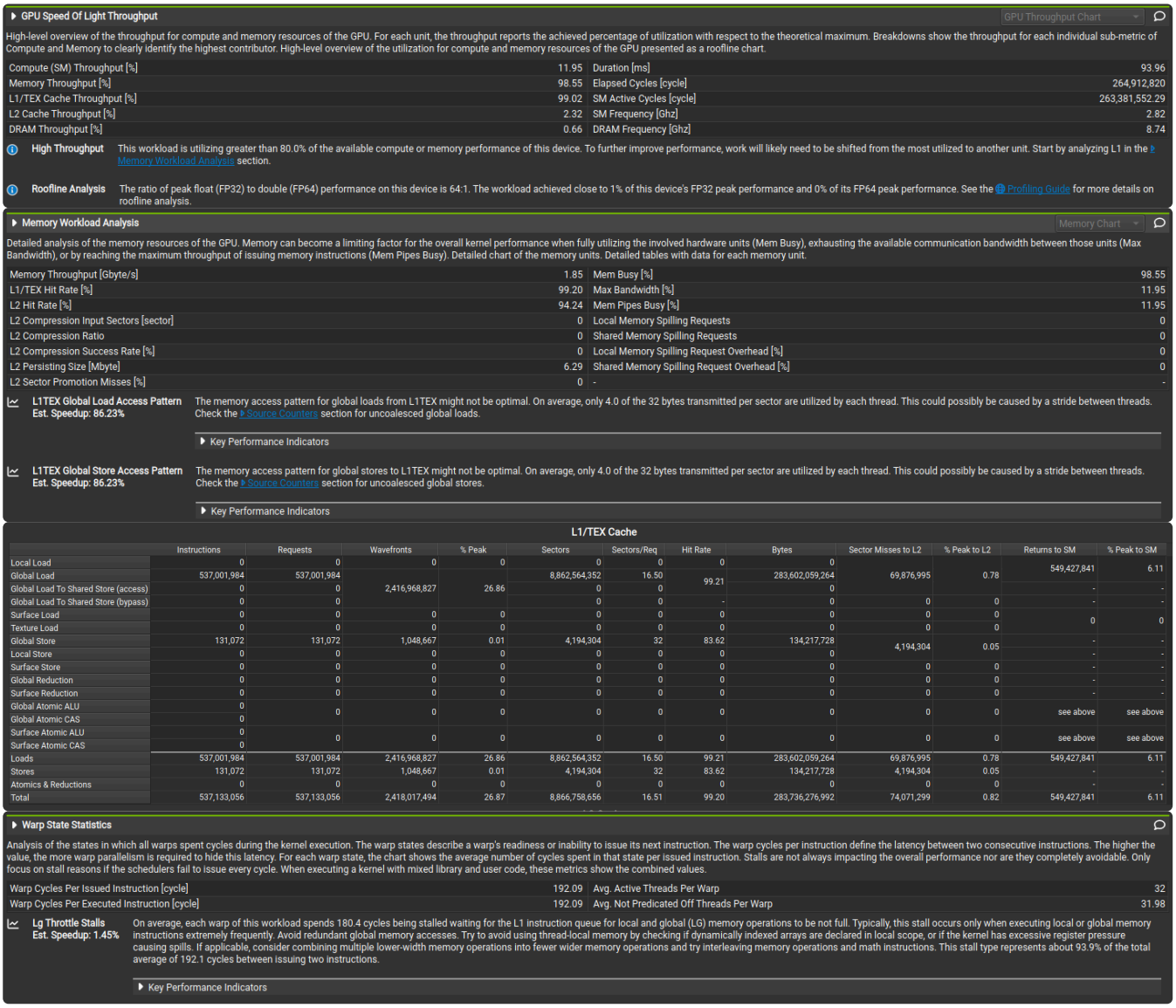
Compute Workload Analysis: issue slots busy 4.08%, executed IPC 0.16.

Conclusion: the kernel is limited by memory access, specifically the L1/TEX path, not by FP32 math.
Memory throughput is at 98.55% (L1/TEX 99.02%) while compute throughput is only 11.95%,
with the issue slots busy only 4% of the time. DRAM (0.66%) and L2 (2.32%) are
almost idle, so this is not a DRAM bandwidth problem: nearly all loads hit in L1/L2,
but the L1 is saturated by the number of sectors it has to process.

Why 16.5 sectors per request: in `naive.cuh`, `threadIdx.x` maps to the **row**, so the 32 lanes of a warp have 32 different rows and the same column.

- `A[row*K + k]`: lanes are K floats apart, so the 32 lanes touch 32 different sectors (32 sectors per request).
- `B[k*N + col]`: all lanes read the same address (a broadcast), so 1 sector per request.
- The average over the two loads is (32 + 1) / 2 = 16.5, which matches the measured value exactly.

Percentage of peak FP32: 2 x 2048^3 / 93.96 ms = 182.8 GFLOP/s, which is 182.8 / 22,065 = **0.83% of peak FP32** (also 0.83% at 4096^3, using 183.5 GFLOP/s). cuBLAS reaches about 53 to 59% of the same peak.



GPU Speed Of Light Throughput

High-level overview of the throughput for compute and memory resources of the GPU. For each unit, the throughput reports the achieved percentage of utilization with respect to the theoretical maximum. Breakdowns show the throughput for each individual sub-metric of Compute and Memory to clearly identify the highest contributor. High-level overview of the utilization for compute and memory resources of the GPU presented as a roofline chart.

Compute (SM) Throughput [%]	11.95	Duration [ms]	93.96
Memory Throughput [%]	98.55	Elapsed Cycles [cycle]	264,912,820
L1/TEX Cache Throughput [%]	99.02	SM Active Cycles [cycle]	263,381,552.29
L2 Cache Throughput [%]	2.32	SM Frequency [Ghz]	2.82
DRAM Throughput [%]	0.66	DRAM Frequency [Ghz]	8.74

High Throughput This workload is utilizing greater than 80.0% of the available compute or memory performance of this device. To further improve performance, work will likely need to be shifted from the most utilized to another unit. Start by analyzing L1 in the Memory Workload Analysis section.

Roofline Analysis The ratio of peak float (FP32) to double (FP64) performance on this device is 64:1. The workload achieved close to 1% of this device's FP32 peak performance and 0% of its FP64 peak performance. See the Profiling Guide for more details on roofline analysis.

Memory Workload Analysis

Detailed analysis of the memory resources of the GPU. Memory can become a limiting factor for the overall kernel performance when fully utilizing the involved hardware units (Mem Busy), exhausting the available communication bandwidth between those units (Max Bandwidth), or by reaching the maximum throughput of issuing memory instructions (Mem Pipes Busy). Detailed chart of the memory units. Detailed tables with data for each memory unit.

Memory Throughput [Gbyte/s]	1.85	Mem Busy [%]	98.55
L1/TEX Hit Rate [%]	99.20	Max Bandwidth [%]	11.95
L2 Hit Rate [%]	94.24	Mem Pipes Busy [%]	11.95
L2 Compression Input Sectors [sector]	0	Local Memory Spilling Requests	0
L2 Compression Ratio	0	Shared Memory Spilling Requests	0
L2 Compression Success Rate [%]	0	Local Memory Spilling Request Overhead [%]	0
L2 Persisting Size [Mbyte]	6.29	Shared Memory Spilling Request Overhead [%]	0
L2 Sector Promotion Misses [%]	0	-	-

L1TEX Global Load Access Pattern Est. Speedup: 86.23%
The memory access pattern for global loads from L1TEX might not be optimal. On average, only 4.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Source Counters section for uncoalesced global loads.

Key Performance Indicators

L1TEX Global Store Access Pattern Est. Speedup: 86.23%
The memory access pattern for global stores to L1TEX might not be optimal. On average, only 4.0 of the 32 bytes transmitted per sector are utilized by each thread. This could possibly be caused by a stride between threads. Check the Source Counters section for uncoalesced global stores.

Key Performance Indicators

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to L2	Returns to SM	% Peak to SM
Local Load	0	0	0	0	0	0	0	0				
Global Load	537,001,984	537,001,984			8,862,564,352	16.50	99.21	283,602,059,264	69,876,995	0.78	549,427,841	6.11
Global Load To Shared Store (access)	0	0	2,416,968,827	26.86	0	0		0			-	-
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	0	-	-
Surface Load	0	0	0	0	0	0	0	0	0	0		
Texture Load	0	0	0	0	0	0	0	0	0	0	0	0
Global Store	131,072	131,072	1,048,667	0.01	4,194,304	32	83.62	134,217,728	4,194,304	0.05	-	-
Local Store	0	0	0	0	0	0	0	0			-	-
Surface Store	0	0	0	0	0	0	0	0	0	0	-	-
Global Reduction	0	0	0	0	0	0	0	0	0	0	-	-
Surface Reduction	0	0	0	0	0	0	0	0	0	0	-	-
Global Atomic ALU	0	0	0	0	0	0	0	0	0	0	see above	see above
Global Atomic CAS	0											
Surface Atomic ALU	0	0	0	0	0	0	0	0	0	0	see above	see above
Surface Atomic CAS	0											
Loads	537,001,984	537,001,984	2,416,968,827	26.86	8,862,564,352	16.50	99.21	283,602,059,264	69,876,995	0.78	549,427,841	6.11
Stores	131,072	131,072	1,048,667	0.01	4,194,304	32	83.62	134,217,728	4,194,304	0.05	-	-
Atomics & Reductions	0	0	0	0	0	0	0	0	0	0	-	-
Total	537,133,056	537,133,056	2,418,017,494	26.87	8,866,758,656	16.51	99.20	283,736,276,992	74,071,299	0.82	549,427,841	6.11

Warp State Statistics

Analysis of the states in which all warps spent cycles during the kernel execution. The warp states describe a warp's readiness or inability to issue its next instruction. The warp cycles per instruction define the latency between two consecutive instructions. The higher the value, the more warp parallelism is required to hide this latency. For each warp state, the chart shows the average number of cycles spent in that state per issued instruction. Stalls are not always impacting the overall performance nor are they completely avoidable. Only focus on stall reasons if the schedulers fail to issue every cycle. When executing a kernel with mixed library and user code, these metrics show the combined values.

Warp Cycles Per Issued Instruction [cycle]	192.09	Avg. Active Threads Per Warp	32
Warp Cycles Per Executed Instruction [cycle]	192.09	Avg. Not Predicated Off Threads Per Warp	31.98

Lg Throttle Stalls Est. Speedup: 1.45%
On average, each warp of this workload spends 180.4 cycles being stalled waiting for the L1 instruction queue for local and global (LG) memory operations to be not full. Typically, this stall occurs only when executing local or global memory instructions extremely frequently. Avoid redundant global memory accesses. Try to avoid using thread-local memory by checking if dynamically indexed arrays are declared in local scope, or if the kernel has excessive register pressure causing spills. If applicable, consider combining multiple lower-width memory operations into fewer wider memory operations and try interleaving memory operations and math instructions. This stall type represents about 93.9% of the total average of 192.1 cycles between issuing two instructions.

Key Performance Indicators

Part 2: Coalescing

Change in `src/kernels/coalesced.cuh`: `threadIdx.x` now maps to the **column** and `threadIdx.y` to the row (grid `ceil(N/32) x ceil(M/32)`, block 32x32). The algorithm and the order of loads are unchanged. `check_all.sh` passes for `coalesced` on every shape.

Q1. Benchmark results

Size	naive ms	coalesced ms	speedup	coalesced GFLOP/s	% of cuBLAS
4096^3	748.82	106.92	7.0x	1285	10.4%

At 4096^3 the coalesced kernel reaches 1285 / 22,065 = 5.8% of peak FP32 (naive: 0.83%).

```
GPU: NVIDIA GeForce RTX 4060 Ti  (sm_89, 34 SMs, L2 32.0 MB)
Approx. peaks: FP32 22326 GFLOP/s, DRAM 288 GB/s  (verify against your spec sheet!)
Problem: M=4096 N=4096 K=4096 alpha=1 beta=0

id  kernel                          ms     GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
2   coalesced                 106.9210      1285.4       1.9    10.4%     5.8%     0.7%  PASS (err 0.0e+00)
```

Q2. Why did the speed change?

ncu at 2048^3 (global loads, same 537,001,984 requests in both kernels):

Metric	naive	coalesced
Global-load sectors	8,862,564,352	1,342,701,568
Sectors per request	16.5	**2.5**
Duration (ncu)	93.96 ms	11.42 ms
L1/TEX throughput	99.02%	98.89%
Dominant stall	LG throttle (180 of 192 cycles)	LG throttle (17.7 of 23.5 cycles)
Cycles between issued instructions per warp	192.1	23.5

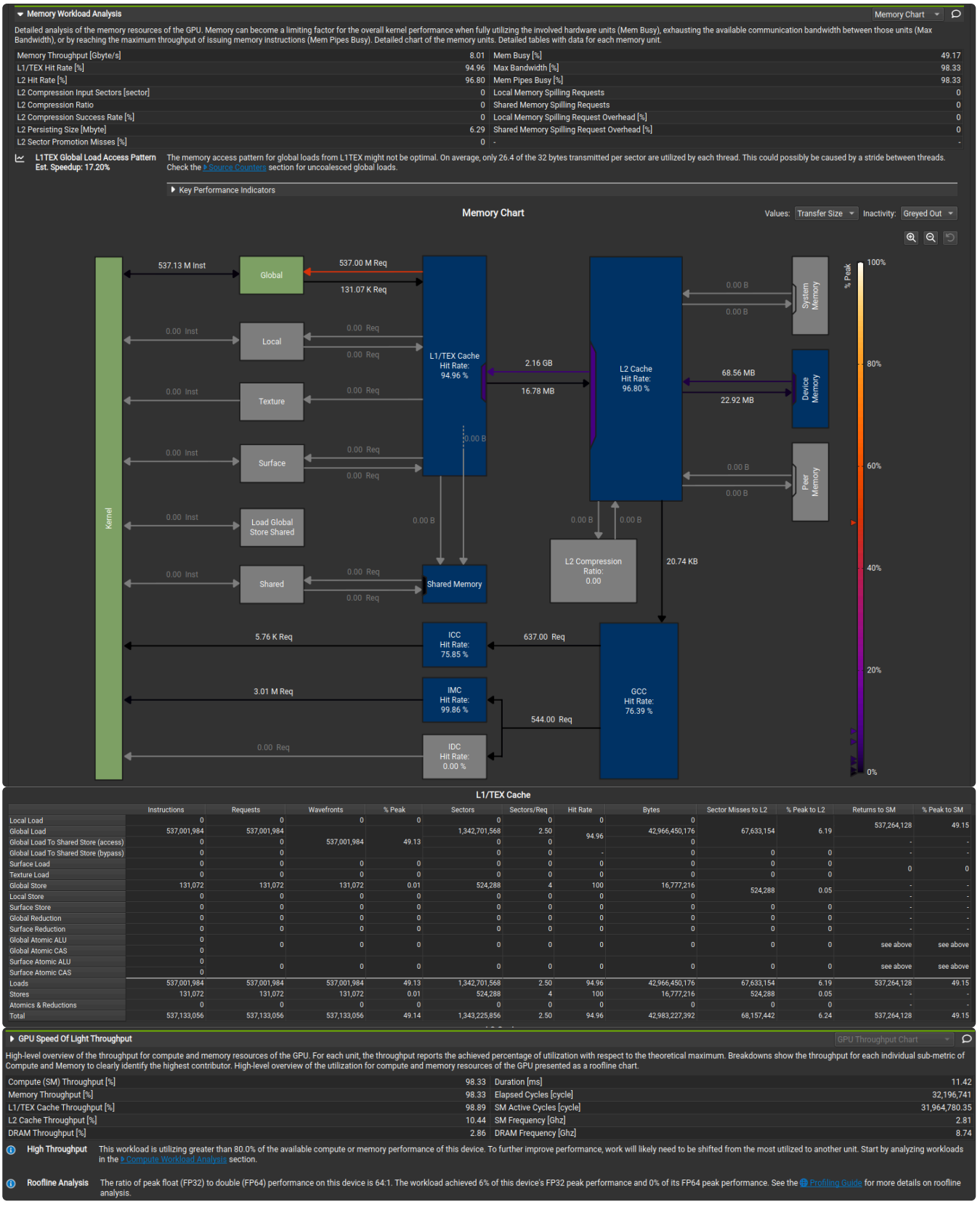
The number of requests (one per warp-level load instruction) and the FLOPs are identical. What changed is how many 32-byte sectors the L1 must process per request.

- Before: lanes had different rows, so `A[row*K+k]` hit 32 sectors per request and `B[k*N+col]` (same column) hit 1. Average 16.5.
- After: lanes have consecutive columns. `B[k*N+col]` reads 32 consecutive floats = 128 bytes = 4 sectors, and `A[row*K+k]` is a broadcast of one address = 1 sector. Average (4 + 1) / 2 = **2.5**, which matches the measurement.

The L1/TEX unit processes sectors at a fixed rate, and it was saturated in the naive kernel. With 6.6x fewer sectors for the same work, each warp waits far less for the L1 queue (192 down to 23.5 cycles per issued instruction), so the kernel runs about 8x faster. The L1 is still at about 99% of its throughput, so the kernel is still limited by L1 traffic, just much less so.

Q3. Which memory level is busiest, and where do the repeated loads go?

- Busiest: **L1/TEX at 98.9%**. In the Memory Chart the red arrow between the Global unit and the L1/TEX cache (537.00 M requests) is the one near 100% of peak, while the links to L2 and device memory are dark purple (low utilization). L2 is at 10.4% and DRAM at 2.9%.
- The repeated loads (each element of A is loaded N times and each element of B M times) are served from the **L1 cache and then the L2 cache, not DRAM**:
  - L1/TEX hit rate is 94.96%.
  - Of the requests that miss L1, the L2 hit rate is 96.8%. L2 receives 67.6M sector reads (about 2.2 GB) against 1.34 billion sectors (about 43 GB) requested from L1.
  - DRAM reads only 68.6 MB, close to the 3 x 16.8 MB minimum for the three 2048^3 matrices, which means each element of A and B really is fetched from DRAM roughly once.
- So the data reuse is real, but it is happening through hardware caches. Every one of the 2N^3 loads still goes through the L1, which is why L1 stays saturated. Part 3 fixes this by keeping tiles in shared memory and registers explicitly.



Part 3: Tiling with shared memory and registers

Implemented `sgemm_smem<TILE>` (`smem16`, `smem32`) and `sgemm_reg2d` in `src/kernels/`. Six reg2d configurations are registered in `src/kernels.cuh`.
All pass the full `check_all.sh` including the sanitizer stage (203 PASS, 0 FAIL; memcheck and racecheck clean on every shape).

Q1. Results

Benchmark at 4096^3 (cuBLAS: 11.14 ms, 12,337 GFLOP/s):

Kernel	ms	GFLOP/s	% cuBLAS
smem16	81.29	1,691	13.7%
smem32	83.29	1,650	13.4%
reg2d_128x128x8_8x8	15.55	8,839	71.6%
reg2d_128x128x16_8x8	**15.34**	**8,959**	**72.6%**
reg2d_128x64x8_8x4	18.09	7,599	61.6%
reg2d_64x64x32_4x4	24.04	5,716	46.3%
reg2d_128x128x8_4x4	24.42	5,628	45.6%
reg2d_64x64x8_4x4	25.54	5,381	43.6%

```
GPU: NVIDIA GeForce RTX 4060 Ti  (sm_89, 34 SMs, L2 32.0 MB)
Approx. peaks: FP32 22326 GFLOP/s, DRAM 288 GB/s  (verify against your spec sheet!)
Problem: M=4096 N=4096 K=4096 alpha=1 beta=0

id  kernel                          ms     GFLOP/s      GB/s  %cuBLAS   %peakF  %peakBW  check
0   cublas                     11.1407     12336.6      18.1   100.0%    55.3%     6.3%  PASS (err 0.0e+00)
1   naive                     766.4732       179.3       0.3     1.5%     0.8%     0.1%  PASS (err 0.0e+00)
2   coalesced                 106.8349      1286.5       1.9    10.4%     5.8%     0.7%  PASS (err 0.0e+00)
3   smem16                     81.2945      1690.6       2.5    13.7%     7.6%     0.9%  PASS (err 0.0e+00)
4   smem32                     83.2864      1650.2       2.4    13.4%     7.4%     0.8%  PASS (err 0.0e+00)
5   reg2d_128x128x8_8x8        15.5491      8839.1      12.9    71.6%    39.6%     4.5%  PASS (err 0.0e+00)
6   reg2d_64x64x8_4x4          25.5416      5381.0       7.9    43.6%    24.1%     2.7%  PASS (err 0.0e+00)
7   reg2d_128x64x8_8x4         18.0869      7598.8      11.1    61.6%    34.0%     3.9%  PASS (err 0.0e+00)
8   reg2d_128x128x16_8x8       15.3405      8959.2      13.1    72.6%    40.1%     4.6%  PASS (err 0.0e+00)
9   reg2d_64x64x32_4x4         24.0446      5716.0       8.4    46.3%    25.6%     2.9%  PASS (err 0.0e+00)
10  reg2d_128x128x8_4x4        24.4228      5627.5       8.2    45.6%    25.2%     2.9%  PASS (err 0.0e+00)
11  custom_pad                 14.6884      9357.0      13.7    75.8%    41.9%     4.8%  PASS (err 0.0e+00)
12  custom_pad4                14.9135      9215.7      13.5    74.7%    41.3%     4.7%  PASS (err 0.0e+00)
13  custom_cf                  14.2513      9643.9      14.1    78.2%    43.2%     4.9%  PASS (err 0.0e+00)
14  custom_vec4                12.7692     10763.3      15.8    87.2%    48.2%     5.5%  PASS (err 0.0e+00)
15  custom_cf_pad4             14.9725      9179.4      13.4    74.4%    41.1%     4.7%  PASS (err 0.0e+00)
```

Experiment table (ncu `--set full` at 4096^3, `--clock-control none`; "time" is the ncu kernel duration, so it is slightly lower than the harness median).
Theoretical occupancy is `sm__maximum_warps_per_active_cycle_pct`, achieved is `sm__warps_active.avg.pct_of_peak_sustained_active`.
Bank conflicts are `l1tex__data_bank_conflicts_pipe_lsu_mem_shared_op_{ld,st}`, in millions.
The dominant stall is the largest `warps_issue_stalled_*_per_issue_active` ratio, excluding `selected` and `not_selected`.

Config	Time (ms)	Regs/thr	Smem/block	Occ. theor.	Occ. achieved	Dominant stall (ratio)	Conflicts ld	Conflicts st
128x128x8, 8x8	15.10	120	8.2 KB	33.3%	33.3%	Long scoreboard (1.28)	268M	117M
128x128x16, 8x8	14.87	125	16.4 KB	33.3%	33.3%	Long scoreboard (1.54)	268M	252M
128x64x8, 8x4	17.63	80	6.1 KB	50.0%	49.6%	Long scoreboard (3.70)	0	235M
64x64x32, 4x4	23.13	64	16.4 KB	66.7%	66.1%	MIO throttle (5.43)	0.2M	**1,040M**
128x128x8, 4x4	24.29	64	8.2 KB	66.7%	66.7%	Long scoreboard (7.04)	0	123M

Config	Time (ms)	Regs/thr	Smem/block	Occ. theor.	Occ. achieved	Dominant stall (ratio)	Conflicts ld	Conflicts st
64x64x8, 4x4	25.49	58	4.1 KB	66.7%	66.2%	Long scoreboard (7.81)	0	235M

Registers and shared memory come from the `make` ptxas output. The 128x128 4x4 config uses 1024 threads per block, so only one block fits per SM.

Q2. Shared-memory loads per FMA

- **3a (`smem` , one output per thread):** each FMA needs `As[ty][k]` and `Bs[k][tx]` , which is **2 shared loads per FMA**.
- **3b (reg2d, TM = TN = 8):** for each k a thread loads TM + TN = 16 values (8 of A, 8 of B) and does TM x TN = 64 FMAs, which is **16 / 64 = 0.25 loads per FMA**, 8x fewer than 3a.

The FMA units are idle while a thread waits for shared-memory data, and the shared-memory bandwidth (the LSU pipe) is limited.
In 3a the kernel spends most of its time issuing loads for every FMA.
In 3b each value loaded into a register is reused 8 times, so the same shared-memory bandwidth feeds 8x more FMAs, and the kernel becomes much less bound by shared-memory access.
That is the jump from about 1.6 TFLOP/s (smem) to about 8.5 TFLOP/s (reg2d 8x8), a 5.2x speedup.
A 4x4 thread tile gives 8 / 16 = 0.5 loads per FMA, which is only a 4x reduction, and those configs are indeed slower.

Q3. Fastest configuration and a slower one

Fastest: `reg2d_128x128x16_8x8` (15.34 ms, 72.6% of cuBLAS), with `128x128x8_8x8` essentially tied (15.55 ms).
These have the lowest shared loads per FMA (0.25) and the most FMAs per thread between barriers.
For this config ncu raises no stall warning: it needs only 6.92 cycles between issued instructions (vs 17.3 for `64x64x8_4x4` and 17.7 for `64x64x32_4x4`), with long scoreboard the largest stall component at 1.54 cycles per issue. They use 120 to 125 registers per thread, which limits them to 2 blocks of 256 threads per SM, 33% theoretical occupancy, but the high instruction-level parallelism of the 64 independent accumulators makes up for it.

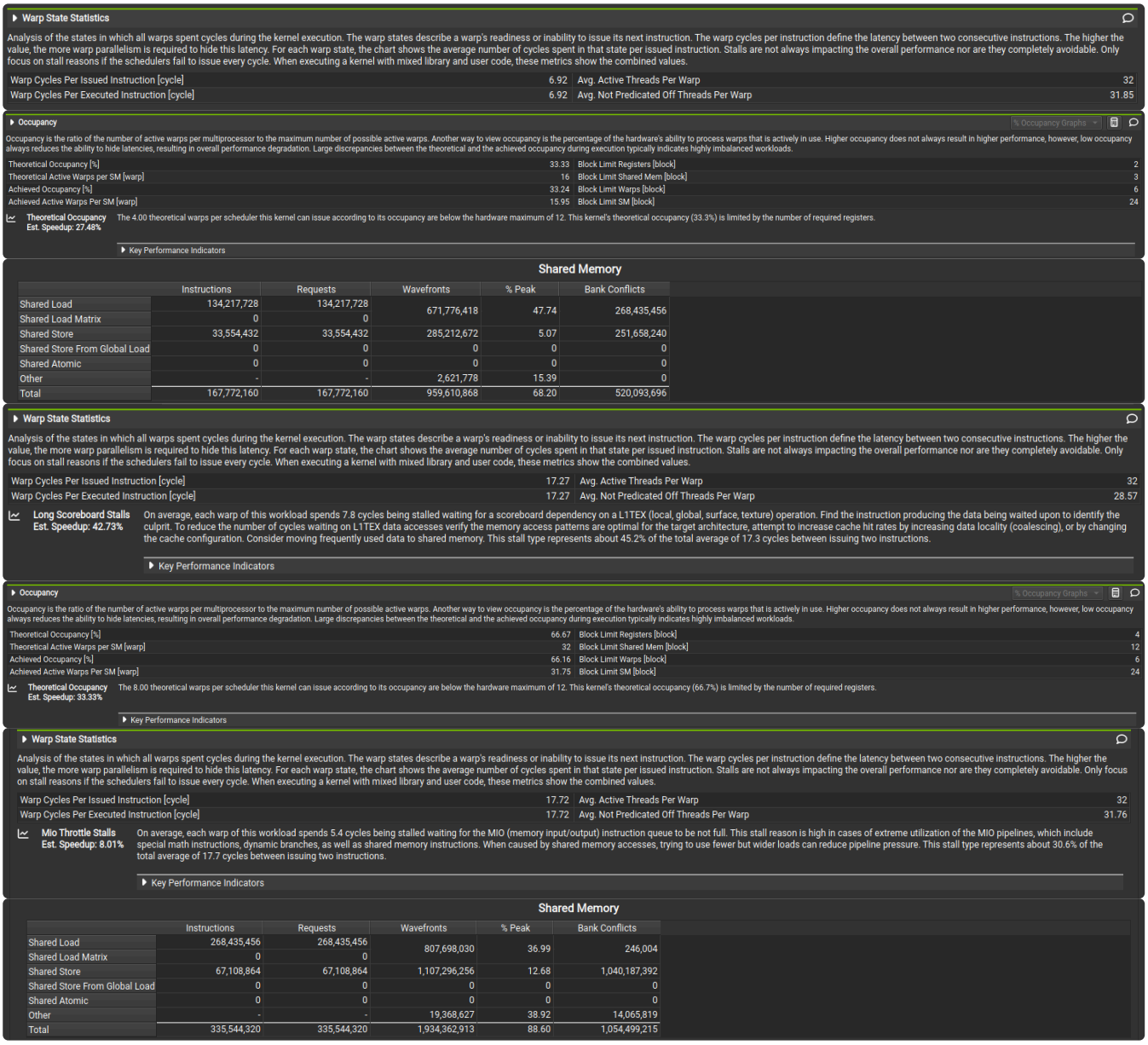
Clearly slower: `reg2d_64x64x8_4x4` (25.49 ms, 1.7x slower than 128x128x8_8x8).
It has *higher* occupancy (66.2% vs 33.3%) and fewer registers (58 vs 120), so occupancy is not the problem. The ncu metrics point to less work per thread:

- Long-scoreboard stall (waiting for global loads) is 7.81 per issued instruction, compared with 1.28 for the fast config. Each thread does only 16 FMAs per k and 128 per tile, so there is far less math between the global loads and the barriers to hide their latency, and extra warps do not make up for it.
- Barrier stalls are also higher (1.50 vs 0.46), because each block does a barrier-synchronized tile load for only 64x64 outputs.
- Shared loads per FMA are 0.5 instead of 0.25, and the shared-load wavefronts are 814M vs 672M.

Another slow one: `reg2d_64x64x32_4x4` (23.13 ms in ncu): bank conflicts.
Its shared-store conflicts are 1,040M (the Shared Memory table), about 9x those of the 128x128x8 8x8 config (117M).
The transposed store `As[k][m]` has consecutive lanes writing consecutive `k` (since `k = idx % BK`) with `m` fixed, so the addresses are `BM` words apart. With BK = 32 and BM = 64 all 32 lanes of a warp hit the same bank (32-way conflict).
The matching ncu signs are MIO throttle (5.43, the top stall) and short scoreboard (3.89), which are both shared-memory pipeline stalls.
The store conflicts grow with BK (117M at BK = 8, 252M at BK = 16 for the 128x128 8x8 configs), so the larger BK that would reduce barriers is partly cancelled by conflicts here.

`reg2d_128x128x8_4x4` is limited by its 1024-thread blocks: one block per SM, with the highest barrier stall ratio (2.82) and long-scoreboard stall (7.04), so there is no other block to run while this one waits at a barrier.

Part 4 can address the conflicts: pad `As` or change the store mapping, and use vectorized loads.



Part 4: Your own optimizations

Chain (`kPart4Chain`): `reg2d_128x128x16_8x8` -> `custom_pad` -> `custom_cf` -> `custom_vec4` .
Timings are from `scripts/grade.sh` at 4096^3 (best of 3 rounds, median of 20 launches).

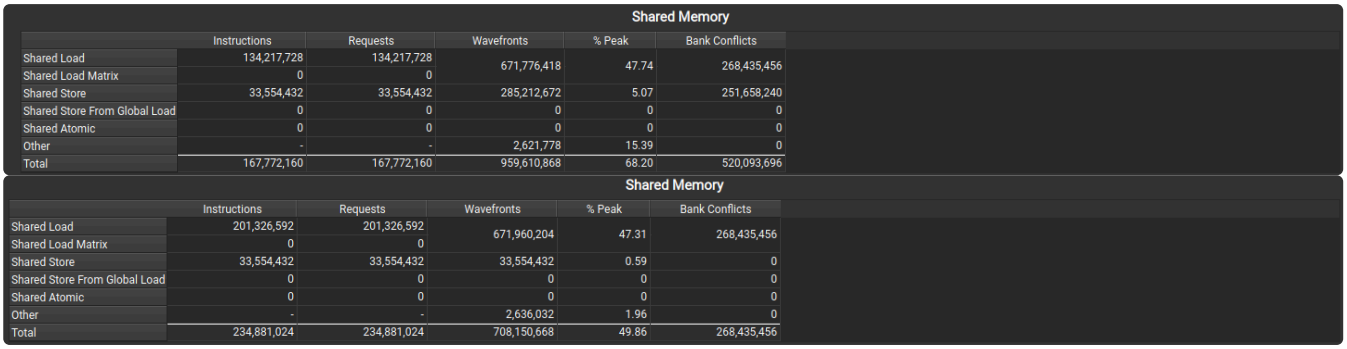
Step 1: padded `As` (`custom_pad` , row stride BM + 2)

- **What I tried:** in the transposed store `As[k][m]` , consecutive lanes write consecutive `k` with the same `m` , i.e. addresses `BM` words apart, which land in the same bank (4-way or worse conflicts, 252M measured by ncu). I padded the row stride from 128 to 130 words (2 odd/even interleaving: lanes with `k` go to even banks and the second `m` to odd banks, so the 32 lanes hit 32 distinct banks).
- **Time (`grade.sh` , final run):** 15.355 ms -> 14.780 ms = **1.039x** (an earlier run gave 15.348 -> 15.061 ms = 1.019x). Both are below the 1.05x threshold, so this step is a failed optimization. Five extra rounds with `--reps 40` gave 1.5% to 2.0%. A pad of 4 gave 0.7% to 1.8% and a pad of 1 was slower (16.35 ms), since with pad 1 half of the lanes still collide.
- **ncu (4096^3, baseline -> custom_pad):**

Metric	reg2d_128x128x16_8x8	custom_pad
Shared-store bank conflicts	251.7M	**0**
Shared-store wavefronts	285.2M	33.6M

Metric	reg2d_128x128x16_8x8	custom_pad
Short-scoreboard stall (per issue)	1.26	0.49
Shared-load bank conflicts	268.4M	268.4M (unchanged)
Shared-load wavefronts	671.8M	672.0M
Duration	14.88 ms	14.70 ms
FMA pipe active	41.3%	42.9%
Instructions executed	3.117G	3.218G (+3%)

- **Why it helped so little:** the padding does what it was meant to do: the store conflicts are gone and the short-scoreboard stall dropped by 60%. But the stores were only about 30% of the shared-memory wavefronts (285M vs 672M for loads, and the stores happen once per tile while the loads happen in every one of the 16 k-steps), so they were not the bottleneck. The main loop is still bound by the 672M shared loads of the outer product (which keep their own 268M conflicts) and by the issue rate (FMA pipe active only about 43%, issue slots about 60% busy). The padded index also costs 3% more instructions. The remaining shared-load conflicts are the next thing to fix.



Shared Memory

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	134,217,728	134,217,728	671,776,418	47.74	268,435,456
Shared Load Matrix	0	0			
Shared Store	33,554,432	33,554,432	285,212,672	5.07	251,658,240
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	2,621,778	15.39	0
Total	167,772,160	167,772,160	959,610,868	68.20	520,093,696

Shared Memory

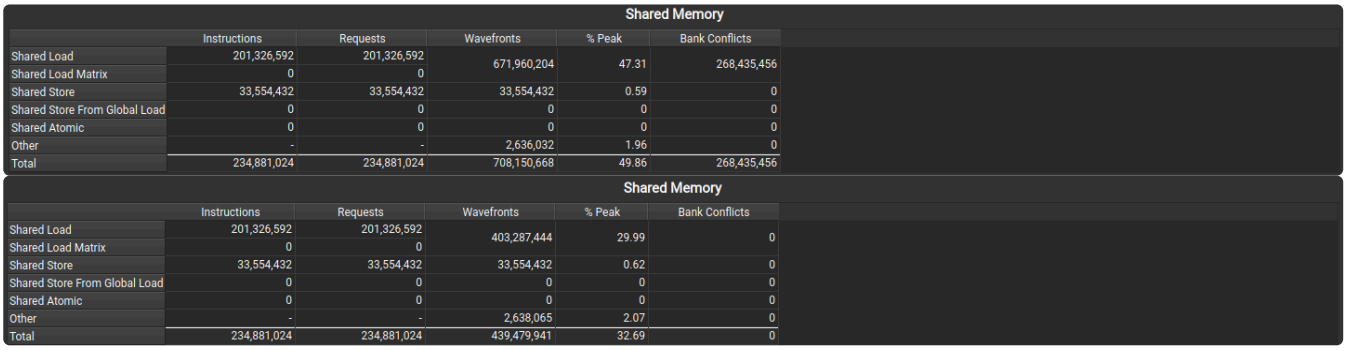
	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	201,326,592	201,326,592	671,960,204	47.31	268,435,456
Shared Load Matrix	0	0			
Shared Store	33,554,432	33,554,432	33,554,432	0.59	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	2,636,032	1.96	0
Total	234,881,024	234,881,024	708,150,668	49.86	268,435,456

Step 2: conflict-free shared loads (`custom_cf`)

- **What I tried:** in the outer-product loop the thread reads its 8 values of B as `Bs[k][tc*8 .. tc*8+7]`. The compiler issues these as two 128-bit loads, and a 128-bit load is served 8 lanes at a time. Those 8 lanes (`tc` = 0..7) read 16-byte chunks 32 bytes apart, so lanes `tc` and `tc+4` hit the same banks (2-way conflict, 268M conflicts measured). I changed which columns a thread owns: instead of 8 contiguous columns it owns two groups of 4 contiguous columns, `tc*4 .. tc*4+3` and `BN/2 + tc*4 .. +3`. Now the 8 lanes read 8 consecutive 16-byte chunks, which is conflict-free. The write-back to C uses the same column mapping, so the results are identical.
- **Time (`grade.sh` , 4096^3):** 14.780 ms -> 14.311 ms = **1.033x** (an earlier run gave 15.045 -> 14.608 ms = 1.030x), below 1.05x, so this step is also a failed optimization. Repeated `--reps 40` runs gave 14.66 to 14.74 ms vs 15.04 to 15.42 ms (2.5% to 4.5%).
- **ncu (4096^3, custom_pad -> custom_cf):**

Metric	custom_pad	custom_cf
Shared-load bank conflicts	268.4M	**0**
Shared-load wavefronts	672.0M	403.3M (-40%)
Duration	14.99 ms	14.30 ms (-4.6%)
FMA pipe active	41.9%	44.3%
Issue slots active	59.8%	62.5%
Short-scoreboard stall (per issue)	0.49	0.40
MIO-throttle stall (per issue)	0.32	0.20
Instructions executed	3.218G	3.218G (unchanged)

- **Why it helped so little:** the conflicts are completely gone and shared-load wavefronts dropped 40%, which lowered the MIO and short-scoreboard stalls. But the kernel is not mainly waiting on shared memory: it executes 3.22G warp instructions for 2.15G FMA warp instructions (only 67% of instructions are FMAs), and the issue slots are already 60 to 62% busy with the FMA pipe at 44%. Removing the conflicts did not change the instruction count, so the gain is limited to the cycles the warps used to stall on the LDS. The remaining stall is long scoreboard (1.37, global-load latency, which each block exposes at every tile boundary), and the remaining non-FMA instructions are index arithmetic and scalar global loads/stores. The next steps should therefore cut instruction count (vectorized `float4` loads) and hide global-load latency (double buffering).



Shared Memory

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	201,326,592	201,326,592	671,960,204	47.31	268,435,456
Shared Load Matrix	0	0			
Shared Store	33,554,432	33,554,432	33,554,432	0.59	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	2,636,032	1.96	0
Total	234,881,024	234,881,024	708,150,668	49.86	268,435,456

Shared Memory

	Instructions	Requests	Wavefronts	% Peak	Bank Conflicts
Shared Load	201,326,592	201,326,592	403,287,444	29.99	0
Shared Load Matrix	0	0			
Shared Store	33,554,432	33,554,432	33,554,432	0.62	0
Shared Store From Global Load	0	0	0	0	0
Shared Atomic	0	0	0	0	0
Other	-	-	2,638,065	2.07	0
Total	234,881,024	234,881,024	439,479,941	32.69	0

Step 3: `float4` global loads (`custom_vec4`)

- **What I tried:** the tile loads used one 32-bit global load per element (`BM*BK + BK*BN` = 4096 loads per block per tile, 16 per thread). I replaced them with 128-bit loads (`float4`). For B the `float4` is stored straight into `Bs` as a `float4` . For A, a `float4` holds 4 consecutive `k` of one row `m` , and I store it transposed into `As[k..k+3][m]` with four scalar stores. With the row padding from step 1 these stores are conflict-free: a warp covers 4 `k` -groups x 8 `m` values, so the banks are `k*8 + m` mod 32 = 32 distinct banks. The launcher returns false if A or B are not 16-byte aligned (all other requirements, M % 128 = N % 128 = K % 16 = 0, are the same as before).
- **Time (`grade.sh` , 4096^3):** 14.311 ms -> 12.855 ms = **1.113x**, a successful optimization (5 points). `--reps 40` runs gave 12.89 to 13.02 ms vs 14.44 to 14.56 ms.
- **ncu (4096^3, custom_cf -> custom_vec4):**

Metric	custom_cf	custom_vec4
Global-load requests	34.08M	**8.91M** (-74%)
Global-load sectors	142.6M	142.6M (same bytes)
Instructions executed	3.218G	**2.797G** (-13%)
FMA pipe active	44.3%	50.0%
Issue slots active	62.5%	63.6%
Registers per thread	125	126
Shared bank conflicts (ld + st)	0	0
Duration	14.30 ms	12.31 ms (-14%)

- **Why it worked:** the bytes moved are identical (142.6M sectors), but each warp-level load request now moves 128 bits per lane, so there are 4x fewer load instructions (34.1M -> 8.9M requests) and the 4x fewer address computations that go with them. That removed 13% of all executed instructions. The kernel is bound by the issue rate (about 63% issue slots busy), so fewer non-FMA instructions turn directly into a higher FMA-pipe utilization (44% -> 50%) and a shorter kernel. Registers did not change, so occupancy stayed the same. The remaining stall is still long scoreboard (1.44 per issue), since the loads for the next tile are issued and waited for at each tile boundary, which is what double buffering would hide.

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to
Global Load	34,078,720	34,078,720			142,606,336	4.18	15.90	4,563,402,752	121,999,720	
Global Load To Shared Store (access)	0	0	34,603,015	2.57	0	0		0		
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	
Surface Load	0	0	0	0	0	0	0	0	0	
Texture Load	0	0	0	0	0	0	0	0	0	
Global Store	524,288	524,288	1,048,576	0.08	8,388,608	16	99.54	268,435,456	2,135,280	
Local Store	0	0	0	0	0	0	0	0		
Surface Store	0	0	0	0	0	0	0	0	0	
Global Reduction	0	0	0	0	0	0	0	0	0	
Surface Reduction	0	0	0	0	0	0	0	0	0	
Global Atomic ALU	0	0	0	0	0	0	0	0	0	
Global Atomic CAS	0									
Surface Atomic ALU	0	0	0	0	0	0	0	0	0	
Surface Atomic CAS	0									
Loads	34,078,720	34,078,720	34,603,015	2.57	142,606,336	4.18	15.90	4,563,402,752	121,999,720	
Stores	524,288	524,288	1,048,576	0.08	8,388,608	16	99.54	268,435,456	2,135,280	
Atomics & Reductions	0	0	0	0	0	0	0	0	0	
Total	34,603,008	34,603,008	35,651,591	2.65	150,994,944	4.36	20.55	4,831,838,208	124,135,000	

L1/TEX Cache

	Instructions	Requests	Wavefronts	% Peak	Sectors	Sectors/Req	Hit Rate	Bytes	Sector Misses to L2	% Peak to
Global Load	8,912,896	8,912,896			142,606,336	16	17.78	4,563,402,752	117,881,296	1
Global Load To Shared Store (access)	0	0	13,656,126	1.19	0	0		0		
Global Load To Shared Store (bypass)	0	0			0	0	-	0	0	
Surface Load	0	0	0	0	0	0	0	0	0	
Texture Load	0	0	0	0	0	0	0	0	0	
Global Store	524,288	524,288	1,048,637	0.09	8,388,608	16	99.19	268,435,456	2,184,380	
Local Store	0	0	0	0	0	0	0	0		
Surface Store	0	0	0	0	0	0	0	0	0	
Global Reduction	0	0	0	0	0	0	0	0	0	
Surface Reduction	0	0	0	0	0	0	0	0	0	
Global Atomic ALU	0	0	0	0	0	0	0	0	0	
Global Atomic CAS	0									
Surface Atomic ALU	0	0	0	0	0	0	0	0	0	
Surface Atomic CAS	0									
Loads	8,912,896	8,912,896	13,656,126	1.19	142,606,336	16	17.78	4,563,402,752	117,881,296	1
Stores	524,288	524,288	1,048,637	0.09	8,388,608	16	99.19	268,435,456	2,184,380	
Atomics & Reductions	0	0	0	0	0	0	0	0	0	
Total	9,437,184	9,437,184	14,704,763	1.28	150,994,944	16	22.30	4,831,838,208	120,065,676	1